User Mode Linux Redesign

Architecture, Implementation State, and Validation Evidence

Reporting cutoff:

`umlctl-deploy` @ `9ccbf5e7bb58`

May 19, 2026

Goal and State

This branch makes the UML execution contract profile-driven.

The proposal keeps UML as Linux built for a host process, then makes backend choice, instrumentation, and validation explicit review surfaces.

backend abstraction

static-key hot paths

profile composition

sustained validation

Preserved Contract and Branch Delta

Preserved UML contract

- upstream Linux built as `arch/um`
- host-process execution model
- rootless/debuggable operation
- no full machine model in scope

Branch-local deltas

- explicit backend-ops contract
- `kvm-v2` trap backend
- static-key instrumentation gates
- profile defconfigs and validation gates

CLAIM change mechanism and policy surfaces, not UML semantics

Branch Goal

Profiles from one tree

- near-native `prod-fast`
- sanitizer-heavy `research`
- KCOV/snapshot `fuzz`
- minimized `sandbox`
- direct-call `library`
- deterministic `time-travel`

Out of scope

- a Linux reimplementation
- a QEMU/Firecracker replacement
- a container runtime
- a from-scratch hypervisor

Adjacent Tool Boundary

Tool	Useful comparison	Boundary for this branch
QEMU/KVM	General virtual machines, device models, booting real images.	This branch keeps the UML host-process kernel model; it is not adding a full machine/device model.
Firecracker	Minimal production microVMs.	It optimizes a different surface: cloud isolation and boot footprint, not `arch/um` profile composition.
gVisor	Userspace kernel sandboxing.	It proves useful backend patterns, but it is not the Linux kernel under test.
LKL	Direct-call Linux library harnesses.	It is fast, but loses the normal user/kernel ring split that several UML profiles preserve.
Containers	Process isolation and deployment.	They share the host kernel; they cannot test another Linux kernel as the target.

Current State at May 19, 2026

Cutoff	`9ccbf5e7bb58` on `umlctl-deploy`
Functional headline	`kvm-v2` guest entry, syscall dispatch, exceptions, signals, SMP state repair, post-gadget performance, snapshot + record/replay primitives, declarative host resource controls
Parity	cpython-parity 21/21 under UP and SMP
Substrate gate	PASS=25/FAIL=3/XFAIL=3, matching seccomp
R14 cache-flake	closed (SMP-T73 YMM-upper XSAVE leak), 220/220 validation soak
Snapshot bench	38.9 μs capture, 13 μs median restore (memo-12 targets $<$50 ms / $<$1 ms)
`umlctl mission`	6/6 phases PASS in $\sim$15 min wall-clock
Benchmark	Faster than seccomp on all eight hosts and all three tiers

DONE foundation + R14 + snapshot/RR + host resources **IN PROGRESS** 24h long-tail soak; Series 7 LKML submission

Inherited UML Versus This Branch

Category	Reading
Inherited UML and Linux	Linux-as-a-host-process, legacy ptrace, generic kernel subsystems, existing KVM APIs, and the broad idea of time travel are foundations, not branch-local inventions.
Upstream prerequisites	Seccomp-mode UML and upstream SMP are enabling context that the redesign builds on.
Redesigned here	Backend ops, static-key gates, profile matrix, `kvm-v2` state ownership, validation ladder, and upstream sequencing.
Implemented here	`arch/um/backend/kvm-v2/` including snapshot (Phase 1–6) and record/replay (Phase 1–7) ports; profile docs and configs; snapshot/forkserver v1; `umlctl` with `mission` subcommand + `[host_resources]` TOML + cgroup v2 + preflight; launcher, gates, soak (11 templates), benchmark, and parity selftests.
Planned or paused	24h long-tail soak; Series 7 LKML submission; record/replay host-side signal wiring; syzkaller `vm/uml` (external); LTP curation; Tier 3 host resources (RT / NUMA / `O_DIRECT`).

DIFF CHECK `origin/master...HEAD` over UML paths: 688 files, 154,501 insertions, 823 deletions.

Evidence Dashboard

21/21
cpython parity
UP and SMP

6/6
`umlctl mission`
acceptance phases

193–908x
micro speedup
over seccomp

7 series
staged upstream
submission queue

Eight Answers: Branch Claims

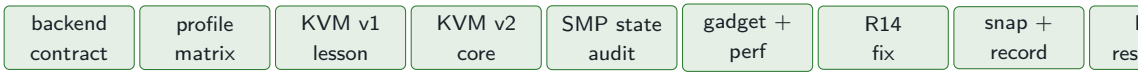
#	Question	Short answer
1	Branch value proposition	One upstream Linux tree can produce fast, research, fuzz, sandbox, library, and deterministic profiles.
2	Adjacent-tool boundary	QEMU, Firecracker, gVisor, LKL, and containers solve different problems; none is the branch target.
3	Shared architecture	Backend ops, static-key gates, and profile defconfigs keep variation explicit.
4	Landed vs planned	Core architecture, `kvm-v2`, snapshot/RR ports, host resource controls, and the `umlctl mission` acceptance gate are in tree; 24h long-tail soak and Series 7 LKML submission remain.

Eight Answers: Review Pressure Points

#	Question	Short answer
5	SMP and state	v2 has explicit state-home contract + audit trail; R14 (YMM-upper XSAVE leak) closed via SMP-T73; T74/T75/T76 latent-bug follow-ups; T77 future-feature checklist.
6	Speed beyond micro	Python + stress tiers win on every benchmark host; snapshot bench 38.9 μs capture and 13 μs median restore.
7	Upstream bar	`umlctl mission` (6/6 phases, $\sim$15 min) is the new acceptance gate; Series 7 cover letter needs refresh + submission.
8	Review pressure	Put pressure on the data model, the 24h long-tail soak, the host-side record/replay signal wiring, and the host_resources cgroup integration.

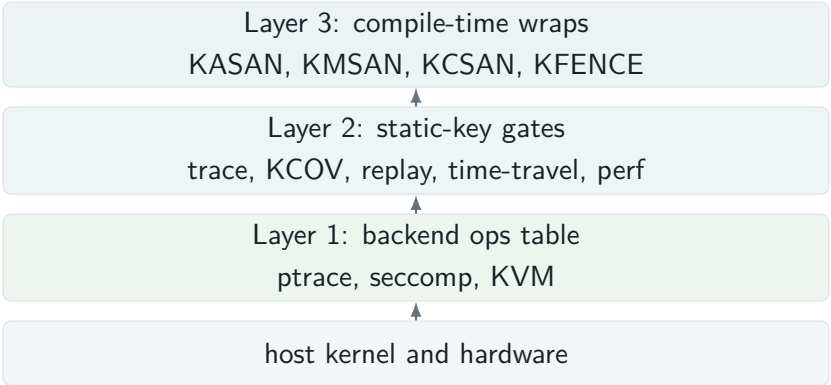
Architecture

Milestone Map



- Three new green lanes since the prior cutoff: R14 closure (SMP-T73), snapshot Phase 1–6 + record/replay Phase 1–7 ports, and host resource controls (SMP-T78–T84).
- The remaining frontier is the 24h long-tail soak and Series 7 LKML submission.

Three-Layer Architecture



Layer 1: Backend Contract

Contract categories

- lifecycle and trap
- memory regions
- scheduling and IPI
- time
- debug/register access

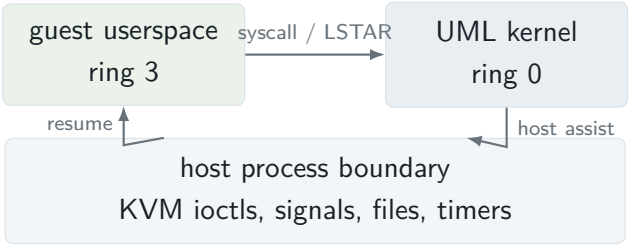
Design point

- every backend implements the table
- single-backend builds can inline calls
- dynamic builds select at boot
- capability flags replace global backend booleans

Profiles Are Product Points

Profile	Backend	Hooks	Role
prod-fast	KVM > seccomp	none	near-native production
prod-with-hooks	KVM > seccomp	compiled, off	production with emergency observability
research	seccomp	trace, kprobes	sanitizer/debug profile
fuzz	seccomp	KCOV, snapshot	syzkaller/harness fuzzing
fuzz-deep	seccomp	KCOV, replay	race and replay investigations
sandbox	seccomp-only	none compiled	minimized TCB
library	direct	n/a	LKL-style harnesses
time-travel	seccomp	deterministic time	replayable execution

KVM v2 Model



This is a trap mechanism for UML, not a general-purpose VMM.

What KVM v1 Taught Us

v1 design pressure

- treat KVM as a faster trap loop
- share too much vCPU/run state
- repair state after exits
- maintain shadow page-table state

lesson

- SMP turns repair workarounds into races
- `kvm_run` must have one active owner
- host signals need KVM-visible masking
- every state item needs an authoritative home

DESIGN SHIFT from “fix the next trap” to an explicit state-ownership contract

Changed Data Model

State class	Authoritative home	Contract in v2
`kvm_run` and vCPU registers	per-host-CPU vCPU	Pick while migration is disabled; no other CPU owns that run page during dispatch.
Guest GPRs and syscall ABI	task registers plus sync-regs mmap	Marshal into KVM before entry and back to UML after exit at explicit boundaries.
FPU/XSAVE	per-task snapshots plus vCPU dirty epoch	Install on first use, capture when ownership changes, avoid cross-task XMM leakage.
IST and exception frames	per-task frame snapshots over per-vCPU pages	Restore before dispatch; snapshot interrupted stubs instead of reading another task's frame.
TLB state	per-mm generation plus per-vCPU last-seen state	Memory changes become generation changes; vCPUs refresh or get kicked.
Host signal delivery	`KVM_SET_SIGNAL_MASK`	`SIGALRM` cannot arbitrarily break the KVM window; a dedicated kick signal can.

The Hard Part Was State Ownership

Audit inputs

- 149 state items
- 50+ operations
- ownership matrix
- suspect register audits
- runtime state trace

Bug classes closed

- migration while using per-CPU vCPU state
- EINTR mid-stub register recovery
- cross-task FPU/XMM leakage
- worker fd leak and shutdown hang
- AVX/XSAVE #UD drift

Validation

Validation Ladder

tooling builds – launcher and `umlctl`

kernel builds – selected profile compiles and links

internal contracts – KUnit marshal/byte-shape; snapshot 3/3; record/replay 7/7

substrate parity – PASS=25/FAIL=3/XFAIL=3, matching seccomp

process/syscall classes – fork, exec, mmap, signals

performance gates – micro/Python/stress + snapshot bench (38.9 μs / 13 μs)

real workload boot – Python and stress workloads

`umlctl mission` – 6/6 phases PASS, 100/100 soak, $\sim$15 min

24-hour long-tail soak – the remaining wall-clock evidence

Mission-Accomplished: Single-Binary Verdict

#	Phase	Verdict	Duration	Summary
1	KUnit	**DONE**	5.0 s	10/10 cases (snap 3 + record 7)
2	Bench	**DONE**	2.5 s	cap 35 μs, median 13 μs
3	Substrate	**DONE**	3.5 s	cpython-tier0 PASS
4	Host resources	**DONE**	1.4 s	UM_THP / UM_OOM_SCORE_ADJ / affinity applied
5	Diverse soak	**DONE**	855 s	100/100 PASS; 0 panics / 0 SIGBUS / 0 high-cr2
6	Diagnostic	**DONE**	0.4 s	kernel/host/cgroup/hugepage metadata

MISSION_ACCOMPLISHED in 867 s (kernel=`9ccbf5e7bb58`)

One command (`umlctl mission --kernel <path>`); JSON scoreboard for CI; `--quick` skips Phase 5 for ~12 s dev iteration.

The R14 Cache-Flake Saga in One Slide

Five weeks, fourteen rounds

- 1–2% bytecode corruption rate in Django soak
- KVMvTwo only; seccomp clean
- Eliminated through R1–R12: KSM, khugepaged, kswapd, host pagecache, fast-page-fault, TLB coherence, mmu_notifier, UML self-unmap, physmem residency, CPython specialization, gadget bounds (T68 shipped as side-effect)
- R13 ruled out MAP_POPULATE pinning as the cause (kept as foundation T71)

Round 14: root cause + fix

- Corruption shape: 16 valid bytes + 16 zero bytes = half AVX2 vector
- SMP-T57 had un-masked AVX in CPUID; XCR0=0x7
- glibc IFUNC picked AVX2 `vmovdqu ymm` memcpy
- Per-task FPU save/restore used legacy `KVM_GET_FPU` (512 B FXSAVE)
- *No YMM upper-128 coverage* $\Rightarrow$ cross-task leak
- Fix (SMP-T73): `KVM_GET_FPU` $\rightarrow$ `KVM_GET_XSAVE` at 5 sites
- 220/220 validation soak; $p = 0.007$

LESSON Three infrastructure follow-ons: T74/T75/T76 latent-bug closures, T77 future-feature checklist, memo 52 host resource controls.

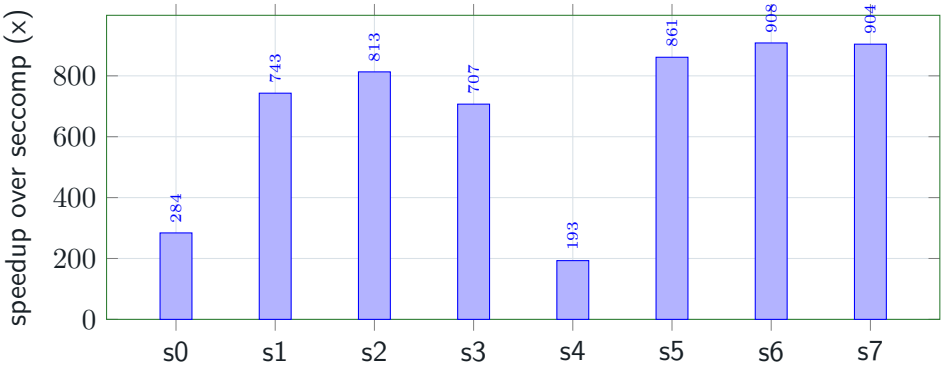
Performance

How To Read the Benchmarks

Tier	What it measures	Why it matters
Micro	RDTSC-bracketed `getpid`-style loop.	Isolates the trap mechanism and LSTAR gadget cost.
Native context	Profile cost model relative to normal Linux `getpid`.	Prevents a speedup-over-seccomp chart from hiding absolute scale.
Python	Startup and mixed-syscall user-land workload.	Checks whether the micro win survives real imports and dynamic linking.
Stress	SMP memory and page-fault-heavy workload.	Checks whether performance survives correctness pressure.

Charts use same-host speedup over seccomp because raw cycles are not portable across CPU generations. The native baseline table gives the human-scale reference point.

Micro Benchmark: The Gadget Changed the Cost Class



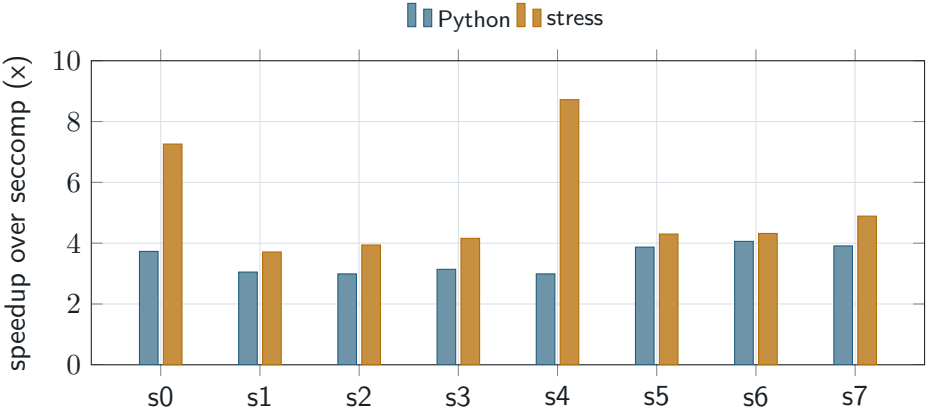
This chart answers only the narrow mechanism question: a gadget-covered trivial syscall no longer pays the seccomp signal/stub round trip.

Native Baseline Context

Path	Relative to normal	Reading
Normal Linux `getpid`	1.0x, 50 ns	Baseline used by the profile cost model.
`kvm-v2 prod-fast`	1.6x, 80 ns	Same order as native for gadget-covered calls.
`seccomp` profiles	10–24x, 505–1200 ns	Expected for research, fuzz, sandbox, and time-travel.
ptrace fallback	40x, 2,000 ns	Compatibility path, not the speed target.
direct library	0.12x, 6 ns	Fast because it removes the ring split.

The cross-host benchmark still reports same-host speedup over `seccomp`; this table provides user-facing native-system context.

Python and Stress Tiers



Python checks userland startup and dynamic-linking behavior; stress checks SMP/page-fault pressure. Both are same-host speedups over seccomp.

Roadmap

Snapshot, Replay, Time Travel

Surface	State	Reading
Snapshot/forkserver v1	DONE	C-09 legacy: `um_snapshot_ready()`, fds 198/199, `state_version`, launcher plumbing, `snapshot-smoke`.
v1 ceiling	LIMIT	status hard-coded zero; no on-disk snapshot; short non-blocking payloads only; no SMP.
KVM-aware snapshot v2 (#168)	DONE	Memo 26 Phase 1–6 in tree: alloc/capture/restore, KUnit 3/3 (basic, full, cross-task), bench harness (38.9 μs / 13 μs), selftest re-plumbed.
KVM-aware record/replay v2 (#169)	DONE	Memo 27 Phase 1–7 in tree: state machine, observe/consume primitives (SYSCALL + RDTSC + SIGALRM), gadget-bypass byte, KUnit 7/7, selftest re-plumbed.
Time-travel profile	IN PROGRESS	UP/seccomp profile exists; runtime hook gate exists; slow path is still a counter stub.
Host signal wiring for replay	PLANNED	Log shape proven by KUnit; remaining work is the host SIGALRM interception that triggers `observe_sigalrm`.

Host Resource Controls (Memo 52)

ID	Lever	Knob
T71	MAP_POPULATE + MAP_LOCKED + mlockall	`UM_KVM_V2_PIN_PHYSMEM=1`
T78	MADV_UNMERGEABLE on physmem	unconditional
T79	MADV_NOHUGEPAGE / MADV_HUGEPAGE	`UM_THP=off / on / auto`
T80	`/proc/self/oom_score_adj` write	`UM_OOM_SCORE_ADJ=N`
T81	MAP_HUGETLB 2M / 1G	`UM_HUGEPAGES=2M / 1G`
T82	`sched_setaffinity` at startup	`UM_KVM_V2_CPU_AFFINITY=list`
T83	cgroup v2 per-instance subgroup	`[host_resources]` TOML in umlctl
T84	preflight verification	automatic when knobs set
T85–T87	SCHED_FIFO RT / NUMA / O_DIRECT	**PLANNED** (deferred per memo 52 §3)

DECLARATIVE `[host_resources]` in any `umlctl .toml`; preflight emits clear stderr warnings for empty hugepage pool / unwritable cgroup / missing CAP_IPC_LOCK

What Remains

Validation

- 24h long-tail wall-clock soak
- Tier 2/Tier 3 template hardening
- LTP curation finalisation
- wider CI matrix

Resumed/external work

- Series 7 LKML cover-letter refresh + submission
- Record/replay host signal wiring
- Snapshot ELF64-core export (#181)
- syzkaller `vm/uml` (external)
- Tier 3 host resources (RT/NUMA/O_DIRECT)

Upstream Path

#	Series	Subsystem	State
1	bpf-hygiene-v1	BPF	ready
2	kmsan-arch-callback-rfc	mm/kmsan	ready
3	ftrace-notrace-generic-v1	tracing	ready
4	backend ops abstraction RFC	arch/um	drafted
5	static-key hot paths	arch/um	drafted
6	per-profile C-series	arch/um + subsystems	blocked
7	KVM backend	arch/um + KVM	cover-letter drafted; *backend-layer unblocked* after R14 + mission gate; needs refresh + submission

Maintainer Review Questions

Architecture

- Is the backend contract minimal?
- Are state homes explicit enough?
- Does `kvm-v2` leak VMM assumptions into UML core?
- Are profile defaults defensible upstream?

Evidence

- Is the validation ladder broad enough?
- Are seccomp comparisons the right control?
- Which LTP failures should be KEEP vs SKIP?
- Are snapshot/replay claims scoped tightly?

The branch is now an implementation with a wall-clock validation frontier.

- Architectural shape is coherent: backends, gates, profiles.
- `kvm-v2` has crossed the hardest feasibility line: real workloads, SMP, speed, snapshot, record/replay, host resource controls.
- The five-week Round-14 cache-flake (YMM-upper XSAVE leak) was root-caused and fixed with infrastructure follow-ons (T74–T77, memo 52).
- A single command — `umlctl mission --kernel <path>` — gives any reviewer a reproducible 15-minute binary verdict.
- Remaining credible milestone: the 24-hour long-tail soak and the Series 7 LKML submission.

Backup: Detailed Speedups

Host	CPU	micro	Python	stress
s0	i9-12900K	284x	3.73x	7.26x
s1	Xeon E3-1225 v6	743x	3.05x	3.71x
s2	Xeon E3-1225 v5	813x	2.99x	3.94x
s3	Xeon W-2123	707x	3.14x	4.16x
s4	i5-12600K	193x	2.99x	8.72x
s5	Ryzen 7 7840HS	861x	3.87x	4.30x
s6	Ryzen 7 7840HS	908x	4.06x	4.32x
s7	Ryzen 7 7840HS	904x	3.91x	4.89x

Backup: Source Map

- `00-vision.md`: goals, non-goals, stakeholders.
- `STATUS.md`: live state, phase ledger, blockers.
- `01-architecture/`: three layers, invariants, data flow.
- `02-workstreams/D-kvm-backend/`: KVM design, v1-to-v2 restart memos, state audit, benchmarks, snapshot+RR port memos (26 + 27).
- `02-workstreams/D-kvm-backend/state-audit/26-feature-enablement-checklist.md`: T77 forward-looking un-mask checklist.
- `08-future-phases/50-...django-flake-investigation-summary.md`: R14 root-cause narrative.
- `08-future-phases/52-uml-host-resource-controls.md`: memo 52 (T78–T84 + T85–T87 deferred).
- `tools/testing/selftests/um/`: gates, benchmarks, soak, parity, snapshot-kvm-smoke, kvm-record-smoke, kvm-snapshot-bench.

Backup: `umlctl mission` CLI

```
$ umlctl mission --kernel <PATH> [--out DIR]
                 [--quick] [--continue-on-fail]
                 [--soak-budget-sec N] [--soak-min-pct PCT]
                 [--selftests-dir DIR]

[Phase 1] PASS kunit       (~5s) -- 10/10 KUnit cases
[Phase 2] PASS bench       (~3s) -- capture=35us median=13us
[Phase 3] PASS substrate   (~4s) -- cpython-tier0 PASS
[Phase 4] PASS host_res    (~1s) -- env knobs applied
[Phase 5] PASS soak        (~14m)-- 100/100 PASS; 0 anomalies
[Phase 6] PASS diagnostic  (<1s) -- metadata captured
MISSION_ACCOMPLISHED in 867s (kernel=9ccbf5e7bb58)
```

JSON scoreboard at `<out>/scoreboard.json`. Exit 0 on ACCOMPLISHED, non-zero with `MISSION_FAILED`

`phase=<N> <name>: <reason>` on any phase failure.